

Aufgabe 1: Drehfreudig

Team-ID: 01004

Team: Wawa

Bearbeiter dieser Aufgabe:
Raphael Porsche

26. August 2025

Inhaltsverzeichnis

Programm-Ausführung.....	1
Lösungsidee.....	2
Umsetzung.....	4
Einlesen.....	4
Drehfreudigkeit überprüfen.....	5
Bild produzieren.....	5
Werkzeuge.....	7
Packages:.....	7
Beispiele.....	7
Quellcode.....	8

Programm-Ausführung

Programmiersprache: Odin Version: dev-2025-02:584fdc0d4

Getestet auf: Linux (ZorinOS 17)

Packages:

- Kernbibliotheken: core:fmt, core:unicode/utf8, core:os, core:strings, core:sort, core:math, core:path/filepath
- vendor:raylib

Benutzungsanweisung:

1. Ein Terminal öffnen.
2. Arbeitsverzeichnis auf das „Aufgabe1_drehfreudig“ Verzeichnis bewegen.
3. Diesen Befehl ausführen: „./drehfreudig.bin“.

Selbstkompilierung:

1. Odin mit den Anweisungen auf <https://odin-lang.org/docs/install/> installieren.

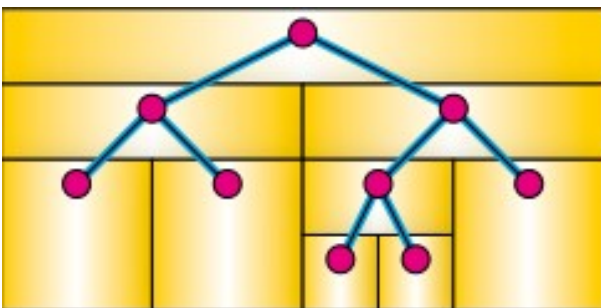
2. Ein Terminal öffnen, das Arbeitsverzeichnis auf das „Aufgabe1_drehfreudig“ Verzeichnis
3. Diesen Befehl ausführen: „odin build drehfreudig“.
4. Nun sollte die ausführbare Datei im „Aufgabe2_choreograph“ Verzeichnis durch eine neue ersetzt sein.

(Q1 – Q8 referenziert verschiedene Teile des Quellcodes)

Lösungsidee

Um herauszufinden ob ein Baum drehfreudig ist, müssen wir uns nur alle Blätter und die Höhe und Breite ihrer Rechtecke anschauen und überlegen, ob die Reihenfolge dieser ‚Größen‘ von links und rechts gelesen dieselbe ist. Anders gesagt müssen sie sich wie ein Palindrom verhalten, wenn man jede Größe in ein Buchstaben umwandeln würde ($1 * 1 \rightarrow A$, $1 * 2 \rightarrow B$... usw.).

Die Höhe aller Knoten haben im Normalfall die gleiche uniforme Maße. Nur Blätterknoten haben in bestimmten Fällen eine größere Höhe, da sich in den gegebenen Bäumen die Blätter in unterschiedlichen Ebenen befinden können und sich höher gelegene Blätter bis auf die Ebene der tiefsten Blätter erstrecken.



Daraus folgend ist die Höhe abhängig von welcher „Tiefe“ sich Blätter befinden, also wie viele Schritte sie von der Wurzel des Baumes entfernt sind. **Gleichung 1** ermöglicht uns die Höhe eines Blattes aus ihrer Tiefe zu berechnen.

Gleichung 1: $h_b = h_u * (t_m - t_b + 1)$

Symbol	Bedeutung
h_b	Die Höhe des behandelten Blattes.
h_u	Die uniforme Normalhöhe eines Knotens.
t_m	Die Tiefe (Abstand zur Wurzel) des tiefsten Blattes.
t_b	Die Tiefe des behandelten Blattes.

Dadurch das h_u (Einheitshöhe) und t_m (maximale Blatt-Tiefe) für jedes Blatt des Baumes konstant sind, ist h_b (Blatt-Höhe) von zwei Blättern zwingend gleich, sobald deren t_b (Blatt-Tiefe) gleich ist. So reicht es aus, die Tiefe von verschiedenen Blättern zu vergleichen um herauszufinden ob deren Höhe gleich ist.

Die Breite eines Knotens ist die Breite des Elternknoten durch deren Anzahl an Kinder, da ein Knoten seine Breite gleichmäßig an seine Kinder in der nächsten „Ebene“ verteilt. **Gleichung 2** drückt dieses Verhalten aus. Um die Breiten in ein uniformes Größenbild zu bringen, benutzen wir eine Prozentzahl. Sie stellt den Anteil der vollen Breite der Darstellung eines einzelnen Knoten dar. Setzen wir jetzt die Wurzel des Baumes auf eine Einheitsbreite von 1, dann ergeben sich die Breiten der Knoten und Blätter durch rekursives Traversieren als Teile eines Ganzen.

Gleichung 2: $b_b = b_e / a_k$

Symbol	Bedeutung
b_b	Breite des betroffenen Knotens.
b_e	Breite des Elternknoten. (Der Knoten über dem jetzigen Knoten)
a_k	Die Anzahl der Kinder des Elternknoten.

Da sowohl Breite als auch Höhe der Rechtecke der Blätterknoten abhängig von allen weiten Knoten sind, ist es sinnvoll zuerst den Baum komplett in eine vorteilhafte Datenstruktur einzulesen. Sie macht es auch einfacher einen Baum letztendlich darzustellen. Hierfür verwenden wir diese Struktur für jeden Knoten: (Q1)

```
node :: struct{
    parent : ^node,
    partition : f32,
    depth : int,
    children : [dynamic]^node,
    visualization : visualization_data
}
```

Ein Knoten (*node struct*) ist folgendermaßen aufgebaut:

- *parent* – einen Zeiger zu seinem Elternknoten (b_e , a_k)
- *partition* – Anteil von der Breite des vollen Baumes (b_b)
- *depth* – die Tiefe des Knoten (t_b) , Schritte zur Wurzel
- *children* – eine Liste von Zeigern zu seinen Kinderknoten

- *visualization* – hält eine weitere Struktur, die wir später benötigen um einen Baum zu zeichnen.

Daraufhin können wir die Partition und Tiefe der Knoten zuteilen. Mit einem rekursiven Einsetzen von **Gleichung 2** können wir die Partition einfach zuweisen und die Tiefe eines Knoten ist jeweils die inkrementierte Tiefe deren Elternknoten.

Um zuletzt die Drehfreudigkeit des eingelesenen Baum dann zu testen, müssen wir den Baum einfach umdrehen und überprüfen ob sich die Dimensionen nicht geändert haben.

Ist nun die Drehfreudigkeit bestätigt, können wir nun ein Bild des drehfreudigen Baumes erstellen. Die Idee ist, ein 1000 * 1000 Pixel Bild mit einem Rechteck und Punkt für jeden Knoten und Linien zwischen den Knoten zu kreieren. Dafür füllen wir für jeden Knoten in unseren Baum die „visualization_data“, die uns beim produzieren eines Bildes hilft. (Q1)

```
visualization_data :: struct {
    pos : [2]rl.Vector2,
    rectangle : [2]rl.Rectangle
}
```

Definition von Vector2 und Rectangle aus raylib:

```
Vector2 :: struct {
    x : f32,
    y : f32
}
Rectangle :: struct {
    x : f32,
    y : f32,
    width : f32,
    height : f32
}
```

Die visualization_data Struktur ist folgendermaßen aufgebaut:

- *pos* – ein Array mit zwei *rl.Vector2*-Elementen, das die jeweilige Mitte des Rechtecks repräsentiert, wo wir den Punkt des Knoten zeichnen. Wir speichern gleich 2 Punkte, da wir den gedrehten Baum gleich mit speichern.
- *rectangle* – ein Array mit zwei *rl.Rectangle*-Elementen, die die Position des Rechtecks und dessen Höhe und Breite so repräsentieren, dass Raylib dieses Rechteck direkt „zeichnen“ kann.

Umsetzung

Einlesen

Für das Einlesen erstellen wir zuerst die Wurzel. Der Wurzel kann direkt ihre Tiefe und Partition von 0 und 1 zugeordnet werden. Außerdem haben wir noch einen weiteren Zeiger, der immer zu dem Knoten zeigt, den wir gerade bearbeiten (Q2)

Nun gehen wir durch die Eingabe Zeichen für Zeichen durch. Ein „(“ bedeutet, wir müssen an den jetzigen Knoten ein weiteres Kind anhängen. Ein „)” bedeutet, dass an dem jetzigen Knoten keine weiteren Kinder angehängt werden müssen. Die Tiefe jedes Knoten können wir dabei auch schon einfach zuordnen, indem jeder neue Knoten einfach die inkrementierte Tiefe deren Elternknoten bekommt. (Q3)

Daraufhin weisen wir die Partition jedes Knotens zu. Dafür wenden wir **Gleichung 2** in einer rekursiven Funktion an jedem Knoten des Baumes an. Wir überspringen die Wurzel, da ihr anfangs schon eine *partition* von 1 zugewiesen wurde. (Q4)

Drehfreudigkeit überprüfen

Um die Drehfreudigkeit zu überprüfen müssen wir zuerst alle Blätter erlangen. Hierfür gehen wir ein weiteres mal durch den Baum und suchen nach allen Knoten, die keine Kinder haben. Dabei achten wir darauf sie in der richtigen Reihenfolge in einer Liste zu sammeln, da diese wichtig ist um die Drehfreudigkeit zu bestimmen. (Q5)

Daraufhin können wir die Drehfreudigkeit feststellen, indem wir durch die Liste der drehfreudigen Blätter mit einem Index i iterieren und überprüfen, ob die Elemente mit der gleichen Distanz von i vom Ende oder Anfang der Liste die Gleiche Breite (*partition*) und die gleiche Tiefe (*depth*) haben. Wenn das für jedes Blatt in der Liste der Fall ist, ist der Baum drehfreudig. (Q6)

Bild produzieren

Wenn ein Bild drehfreudig ist, müssen wir auch ein Bild von diesem erstellen. Hierfür erstellen wir zuerst eine „visualization_data“ Struktur für jeden Knoten und beziehen uns dabei auf die festgelegte Bild-/Fenstergröße von 1000 * 1000 Pixel.

In raylib ist die Position eines Rechtecks immer die obere linke Ecke des Rechtecks. Außerdem hat das Koordinatensystem den Ursprung als obere linke Ecke des Fensters, x wird größer bei einer Bewegung nach rechts und y wird größer bei einer Bewegung nach unten. Die „visualization_data“ Struktur für die Wurzel als Beispiel sieht so aus:

```
visualization_data{
    [2]rl.Rectangle{
        rl.Rectangle{
            x = 0,
            y = 0,
            width = 1000,
            height = 500 / (t_m + 1) # t_m ist die maximale Tiefe.
```

```

    },
    rl.Rectangle{
        x = 1000,
        y = 1000,
        width = -1000,
        height = -500 / (t_m + 1) # t_m ist die maximale Tiefe.
    }
},
[2]rl.Vector2{
    rl.Vector2{
        x = 500,
        y = 250 / (t_m + 1) # t_m ist die maximale Tiefe.
    },
    rl.Vector2{
        x = 500,
        y = 1000 - 250 / (t_m + 1) # t_m ist die maximale Tiefe.
    }
}
}
}

```

Zusammen mit **Gleichung 1-4** gehen wir rekursiv durch den Baum durch und erstellen für jeden weiteren Knoten zwei Rechtecke und zwei Punkte. Ein Rechteck und Punkt für jeweils einer der zwei Bäume (Ein normaler und ein gedrehter Baum). Die Höhe jedes Knoten ist uniform $500/t_m$ mit der Ausnahme von Blätterknoten, bei welchen sie mit **Gleichung 1** errechenbar ist. Die Breite aller Knoten erhalten wir einfach durch die Multiplikation deren *partition* und der vollen Breite von 1000.

Gleichung 3 und **Gleichung 4** geben uns die Position eines Rechtecks. Die Rechtecke jedes gedrehten Teil eines Knoten haben einfach eine Position von $(1000 - x_b, 1000 - y_b)$, eine Breite von $-1 * b_b$ und eine Höhe von $-1 * h_b$. (Q7)

Gleichung 3: $x_b = x_e + b_e * (p_e / n_e)$

Symbol	Bedeutung
x_b	Die x-Position des betroffenen Knotens.
x_e	Die x-Position des Elternknotens.
b_e	Die Breite des Elternknotens.
p_e	Die Position des betroffenen Knoten in den Kindern des Elternknotens.
n_e	Die Anzahl der Kinder des Elternknotens.

Gleichung 4: $y_b = t_b * u_h$

Symbol	Bedeutung
y_b	Die y-Position des betroffenen Knotens.
t_b	Die Tiefe (depth) des betroffenen Knotens.
u_h	Die uniforme Höhe des betroffenen Knotens.

Außerdem erstellen wir noch weitere Punkte für jedes Rechteck aller Knoten. Die x-Position des Punktes errechnet sich durch Addition der x-Position des Rechtecks mit der Hälfte der Breite des Rechtecks: $x_b + (b_b / 2)$. Die y-Position des Mittelpunktes entspricht der y-Position des Rechtecks plus der Hälfte der Höhe des Rechtecks: $y_b + (h_b / 2)$.

Schlussendlich müssen wir ein tatsächliches Bild produzieren. Hierfür zeichnen wir zuerst einen gelben Hintergrund auf ein 1000 * 1000 Pixel großes Fenster mit der Hilfe von raylib. Woraufhin wir ein letztes mal rekursiv durch den Baum durchgehen und die Umrisse der Rechtecke und Punkte malen. Außerdem malen wir noch eine Linie zwischen den Punkten eines Knoten und deren Kinder und lassen die Seite der Blätter aus, die zu der Mitte des Bildes zeigen, wodurch die Blätter sich auf dem gleichen Rechteck wie ihre gedrehten Variationen befinden. Nun machen wir ein Screenshot von dem produzierten Bild und geben es als Ergebnis aus. (Q8)

Werkzeuge

Packages:

core:fmt – Die Ergebnisse ins Terminal zu schreiben, Fehler beim Lesen und Schreiben von Dateien zurückzugeben, um am Ende des Programms die Anzahl von drehfreudigen Bäumen anzuzeigen und Integer als strings zu formatieren.

core:unicode/utf8 – Eingaben strings in eine Liste von Zeichen umwandeln für einfacheren Umgang.

core:os – Schreiben und lesen von Eingaben und Ausgabendateien.

core:strings – Verknüpfen von Ausgaben und teilen von Eingabestrings.

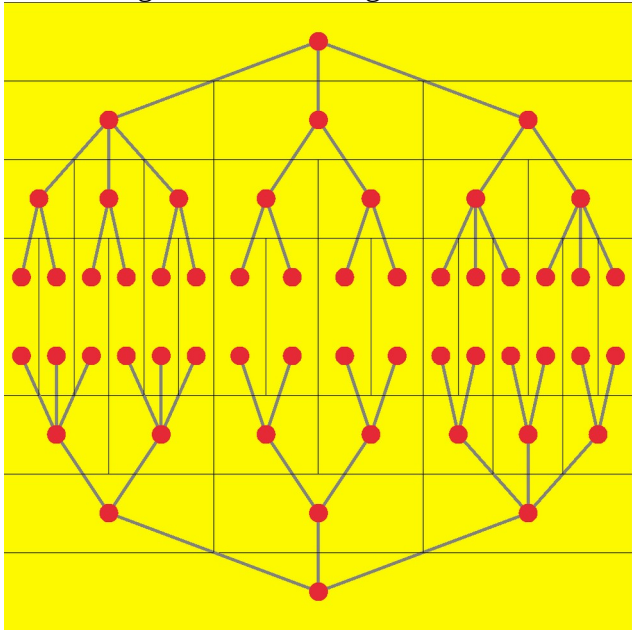
core:sort – Sortieren von Listen um bei der Präsentation der drehfreudigen Bäume sie in der richtigen Reihenfolge zu haben.

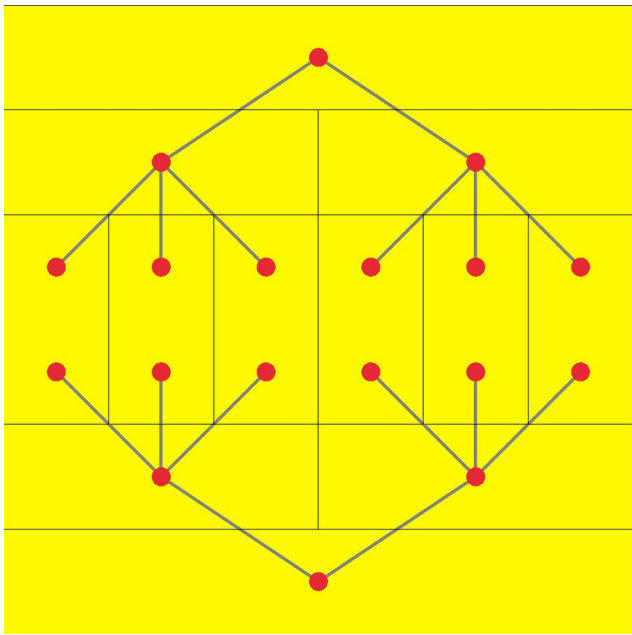
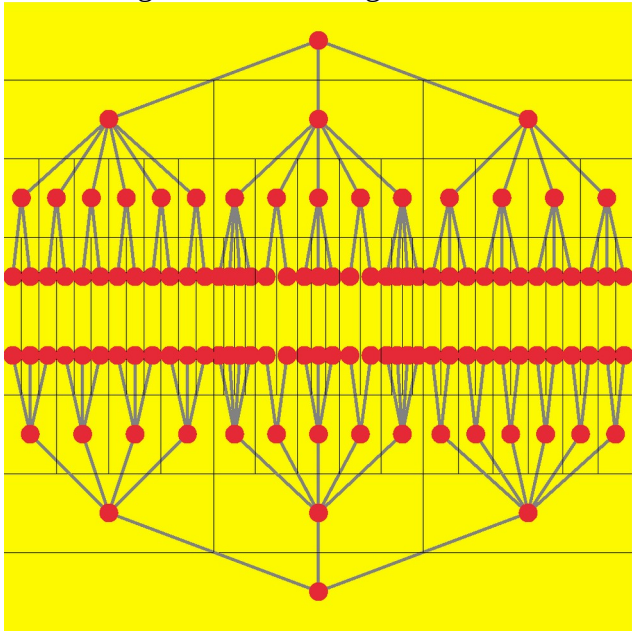
core:math – Um Zehner Potenzen zu berechnen bei der Sortierung der drehfreudigen Bäume nach den Zahlen in ihrem Namen.

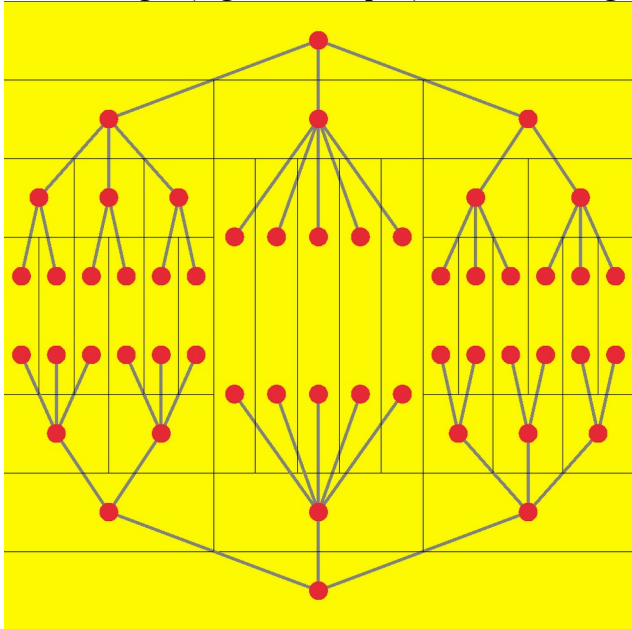
core:path/filepath – Einfacherer und plattformübergreifender Umgang mit Dateieinpfaden.

vendor:raylib – Darstellen der drehfreudigen Bäume auf einem 1000 mal 1000 Pixel Feld. Erlaubt einen durch Funktionen mit bestimmten geometrischen Figuren (Rechtecke, Dreiecke, Kreise, Linien...) verschiedene Bereiche zu färben.

Beispiele

Beispiel	Ausgabe
drehfreudig01.txt	drehfreudig01 ist drehfreudig. 
drehfreudig02.txt	drehfreudig02 ist nicht drehfreudig.
drehfreudig03.txt	drehfreudig03 ist nicht drehfreudig.
drehfreudig04.txt	drehfreudig04 ist drehfreudig.

	
drehfreudig05.txt	drehfreudig05 ist nicht drehfreudig.
drehfreudig06.txt	drehfreudig06 ist nicht drehfreudig.
drehfreudig07.txt	drehfreudig07 ist nicht drehfreudig.
drehfreudig08.txt	drehfreudig08 ist nicht drehfreudig.
drehfreudig09.txt	drehfreudig09 ist nicht drehfreudig.
drehfreudig10.txt	drehfreudig10 ist drehfreudig. 
drehfreudig11.txt	drehfreudig11 ist nicht drehfreudig.

drehfreudig12.txt	drehfreudig12 ist nicht drehfreudig.
drehfreudig13.txt	drehfreudig13 ist nicht drehfreudig.
drehfreudig14.txt	drehfreudig14 ist nicht drehfreudig.
drehfreudig15.txt	drehfreudig15 ist nicht drehfreudig.
drehfreudig16(eigenes_Beispiel).txt	drehfreudig16(eigenes_Beispiel) ist drehfreudig. 

Viele Beispiele sind nicht drehfreudig, obwohl jedes Blatt am Ende die gleiche Breite hat wie das gegenüberliegende Blatt des gedrehten Baumes. Die Beispiele 6, 8, 13 und 15 wären drehfreudig, würde nur die Breite eines Blattes zählen. Dennoch müssen die Rechtecke der Blätter auch in der Höhe gleich sein, so sind auch sie nicht drehfreudig. Dies bestätigt das FAQ mit: „Die Rechtecke von zwei Blättern, die sich direkt gegenüber liegen, wenn man den Baum und den gedrehten Baum passgenau aneinander legt, müssen in Höhe und Breite übereinstimmen.“

Das „ausgaben“ Verzeichnis beinhaltet auch die gezeichneten Bilder von nicht-drehfreudigen Bäumen, dennoch gibt es immer eine .txt Datei, die auf deren Drehfreudigkeit hinweist.

Es gibt kein Beispiel, dass den Sonderfall beinhaltet, wo die Rechtecke der Blätter gleich zu ihrer gedrehten Form sind, also der Baum drehfreudig ist, aber die Höhe der Blätter sich unterscheiden. So wurde beispiel16(eigenes_Beispiel).txt als Beispiel solch einen drehfreudigen Baum hinzugefügt, um auch diesen Sonderfall abzudecken.

Quellcode

Definition der benötigten Strukturen (Q1):

```
node :: struct{
    parent : ^node,
```

```

    partition : f32,
    depth : int,
    children : [dynamic]^node,
    visualization : visualization_data
}
visualization_data :: struct {
    pos : [2]rl.Vector2,
    rectangle : [2]rl.Rectangle
}

```

Erstellen der Wurzel und ‚current_node‘ (Q2):

```

root : ^node = new(node)
root.depth = 0
root.partition = 1
root.children = make([dynamic]^node)
current_node : ^node = root

```

Einlesen des Baumes und Zuweisung der Tiefen (Q3):

```

for char in runeArray {
    if char == '(' {
        if first {
            first = false
            continue
        }
        temp := new(node)
        temp.children = make([dynamic]^node)
        append(&current_node.children, temp)
        temp.depth = current_node.depth + 1
        temp.parent = current_node
        current_node = temp
    } else if char == ')' {
        current_node = current_node.parent
    }
}

```

Ermitteln der Partitionen jedes Knotens (Q4):

```

assign_partition :: proc(nod : ^node, first : bool = false) {
    if !first {
        nod.partition = nod.parent.partition /
f32(len(nod.parent.children))
    }
    for child in nod.children {
        assign_partition(child)
    }
}

```

Finden aller Blätterknoten (Q5):

```

find_leaves :: proc(nod : ^node, leaves : ^[dynamic]^node) {
    if len(nod.children) == 0 {
        append(leaves, nod)
    } else {

```

```

        for child in nod.children {
            find_leaves(child, leaves)
        }
    }
}

```

Drehfreudigkeit überprüfen (Q6):

```

check_drehfreudigkeit :: proc(nod : ^node, leaves : []^node) -> bool {
    for idx in 0..<(len(leaves) / 2) {
        if leaves[idx].partition != leaves[len(leaves) - 1 -
idx].partition {return false}
        if leaves[idx].depth != leaves[len(leaves) - 1 - idx].depth
{return false}
    }
    return true
}

```

Für jeden Knoten eine Darstellung finden (Q7):

```

find_visualization :: proc(nod : ^node, maxdepth : int, layer_height : f32) {
    ypos := layer_height * f32(nod.depth)
    xpos : f32
    if nod.depth != 0 {
        pos_in_parents : int
        for idx in 0..<len(nod.parent.children) {
            if nod.parent.children[idx] == nod {pos_in_parents = idx}
        }
        xpos = nod.parent.visualization.rectangle[0].x + 1000 *
(nod.partition * f32(pos_in_parents))

    } else {
        xpos = 0
    }
    depth_diff_mul : f32 = 1
    if len(nod.children) == 0 {
        depth_diff_mul = f32(maxdepth - nod.depth + 1)
    }
    firstpos := rl.Vector2{xpos + ((nod.partition / 2) * 1000), ypos +
((layer_height * depth_diff_mul) / f32(2))}
    secondpos := rl.Vector2{1000 - (xpos + ((nod.partition / 2) * 1000)), 1000
- (ypos + ((layer_height * depth_diff_mul) / f32(2)))}
    nod.visualization.pos = {firstpos, secondpos}

    firstrectangle := rl.Rectangle{xpos, ypos, 1000 * nod.partition ,
(layer_height * depth_diff_mul)}
    secondrectangle := rl.Rectangle{1000 - xpos, 1000 - ypos, -1000 *
nod.partition, -(layer_height * depth_diff_mul)}
    nod.visualization.rectangle = {firstrectangle, secondrectangle}

    for child in nod.children {
        find_visualization(child, maxdepth, layer_height)
    }
}

```

Darstellung des Baums (Q8):

```
visualize :: proc(nod : ^node) {
  for i in 0..2 {
    temp := &nod.visualization.rectangle[i]
    rl.DrawLineV(rl.Vector2{temp.x, temp.y}, rl.Vector2{temp.x,
temp.y + temp.height}, rl.BLACK)
    rl.DrawLineV(rl.Vector2{temp.x + temp.width, temp.y},
rl.Vector2{temp.x + temp.width, temp.y + temp.height}, rl.BLACK)
    rl.DrawLineV(rl.Vector2{temp.x, temp.y}, rl.Vector2{temp.x +
temp.width, temp.y}, rl.BLACK)
    if !(len(nod.children) == 0) {
      rl.DrawLineV(rl.Vector2{temp.x, temp.y + temp.height},
rl.Vector2{temp.x + temp.width, temp.y + temp.height}, rl.BLACK)
    }
    for child in nod.children {
      rl.DrawLineEx(child.visualization.pos[i],
nod.visualization.pos[i], 5, rl.GRAY)
      if i == 1 {visualize(child)}
    }
    rl.DrawCircleV(nod.visualization.pos[i], 15, rl.RED)
  }
}
```